kc: what a PTX compiler must do itself

A technical report on kc, a small compiler from a CUDA-like language to NVIDIA PTX, and on measuring it against nvcc. October 2026.

Summary. kc is a compiler written from scratch in C++ (about 4,700 lines, no LLVM) that turns a small GPU kernel language into PTX, NVIDIA's virtual GPU assembly. PTX is then compiled to machine code (SASS) by ptxas. Because ptxas is itself an optimizing compiler, a PTX producer has to decide which optimizations to do itself and which to leave to ptxas. I measured this boundary: ptxas already performs common subexpression elimination, loop-invariant code motion, constant folding and dead code removal, so kc's versions of those left the machine code byte-for-byte unchanged. Three optimizations, however, are left to the front end: **inlining**, **loop unrolling** and **division by constants**. With those three added, kc runs within 6% of nvcc, and up to 6% faster, on all seven benchmark kernels, with both compilers' PTX going through the same ptxas. Random differential testing of 2,050 generated programs found no kc bugs, but it did find a ptxas optimizer bug that nvcc's output never triggers.

1. Background and question

A GPU program reaches the hardware in two steps. A front end (nvcc's NVVM stage, OpenAI Triton, Julia, Numba, ...) writes **PTX**: a stable, virtual instruction set with unlimited registers. Then **ptxas**, either offline or inside the GPU driver, compiles PTX into **SASS**, the machine code of one GPU generation, doing register allocation, scheduling and many optimizations.

This raises a practical question for anyone who writes a PTX producer: which optimizations are worth implementing before PTX, and which does ptxas already do? I built kc to answer it by experiment, on an RTX 4060 Laptop GPU (sm_89).

2. The compiler

```
source -> lexer -> parser -> type checker -> SSA IR -> divergence check -> optimizer -> PTX -> ptxas -> SASS
```

- **Language.** Kernels and device functions (with recursion), `i32` / `f32` / `bool` and pointers, `if` / `while` / `for` / `break` / `continue`, shared memory and `barrier()`, atomics, and warp operations (`warp_sum`, `shfl_down`, `shfl_xor`, `ballot`, `lane_id`). Types are strict; there are no implicit conversions.
- **Front end.** A hand-written lexer, a recursive-descent parser with precedence climbing, and a type checker with line:column errors.
- **SSA IR.** Built in one pass from the syntax tree with the algorithm of Braun et al. (2013), so every value is assigned exactly once. A verifier checks the SSA rules (for example, that every definition dominates its uses) after IR generation and after every optimization pass.

- **Analyses.** Dominators and post-dominators (Cooper, Harvey and Kennedy, 2001), natural loops, and a divergence analysis that finds values which can differ between threads. It warns when `barrier()` or a warp operation is reached by only some threads, and marks uniform branches `bra.uni`.
- **Optimizer (-O1).** Inlining, constant folding (bit-exact with the GPU's arithmetic), strength reduction of division by powers of two, common subexpression elimination over the dominator tree, loop-invariant code motion, dead code elimination and 4x loop unrolling.
- **PTX backend.** Phi elimination with one "mailbox" register per phi (correct even when phis read each other), the `.param` calling convention, constants as immediate operands, per-architecture lowering (`warp_sum` is one `redux.sync` on sm_80 and newer, five butterfly shuffles before), `.loc` line info, and `.extern` / `.visible` functions for separate compilation.
- **Runtime.** The CUDA Driver API JIT-loads PTX; the PTX Compiler API (nvPTXCompiler) and nvJitLink compile and link PTX inside the process.

Component	Lines of C++
lexer, parser, type checker	1,471
IR and SSA construction	925
analyses, verifier, divergence	584
optimizer (7 passes)	1,080
PTX backend	467
tests	1,172

3. Method

Same ptxas for both. Seven kernels were written in kc and, line for line, in CUDA C++ (the same algorithm, no `__restrict__`). nvcc 13.3 writes PTX version 9.3, which the installed 13.1 driver rejects (`CUDA_ERROR_UNSUPPORTED_PTX_VERSION`), so both compilers' PTX was compiled by the same library, **nvPTXCompiler 13.3**, at its default `-O3`. Any speed difference therefore comes from the PTX, not from different versions of ptxas.

Checked, then timed. Every version first had to reproduce a CPU reference result. Each kernel was then timed with CUDA events: the median of 5 runs, each the average of 20 back-to-back launches, on 16M elements (matmul: 512 x 512). On this laptop GPU, repeated runs on an otherwise idle machine vary by about 2-3%, and three separate runs agreed. A laptop GPU's clock depends on power and temperature: a later run made while other programs were using the GPU was erratic (some kernels 3-10x slower, in no consistent direction). So the benchmark should be run on an idle, plugged-in machine.

Machine code. SASS was compared with `cuobjdump -sass` (instruction counts, and byte-for-byte diffs between `-O0` and `-O1`), and register counts come from `ptxas -v`.

4. Results

4.1 kc matches nvcc

kernel	kind	kc -O0	kc -O1	nvcc	kc -O1 / nvcc
saxpy	memory bound	0.923 ms	0.927 ms	0.920 ms	1.01
row_sum	loop per thread	0.811 ms	0.517 ms	0.520 ms	0.99
matmul	nested loops	0.499 ms	0.340 ms	0.361 ms	0.94
block_sum	shared-memory reduction	0.655 ms	0.561 ms	0.560 ms	1.00
warp_reduce	warp_sum + atomics	0.285 ms	0.283 ms	0.285 ms	0.99
histogram	global atomics	1.424 ms	1.422 ms	1.423 ms	1.00
adjust	two device functions	0.580 ms	0.595 ms	0.592 ms	1.01

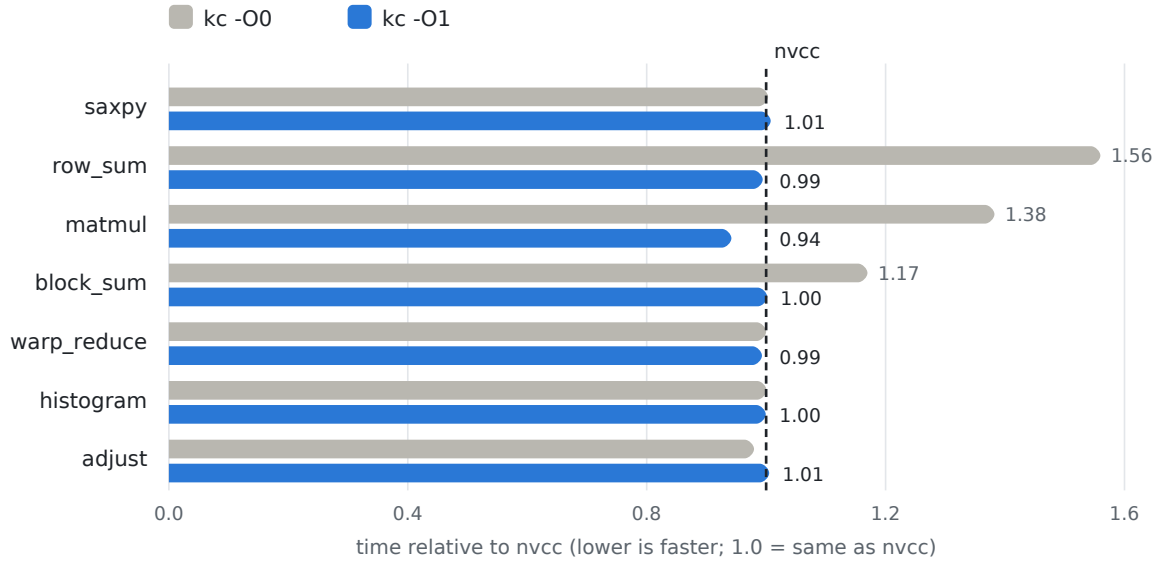


Figure 1. Time relative to nvcc. Without optimization (`-O0`), the loop kernels are 1.2-1.6x slower; with `-O1`, every kernel is within the 2-3% run-to-run noise of nvcc, or faster (matmul).

4.2 What ptxas already does

With the first version of the optimizer (inlining, folding, CSE, LICM, DCE), the SASS at `-O0` and `-O1` was **byte-for-byte identical** for matmul, row_sum, saxpy and grid_stride, and reduce differed only in register numbers. ptxas had already removed the duplicate address computations, hoisted `r * cols` out of loops and folded constants. It also merges the phi "mailbox" moves during register allocation, turns small `if`s into predicated instructions (relu: 25 PTX instructions become 15 SASS, with no branch), turns atomics whose result is unused into `RED`, and combines warp-uniform constant atomics into one `RED` per warp. Marking uniform branches `bra.uni` changed nothing either: ptxas runs its own uniformity analysis.

4.3 What ptxas leaves to the front end

Reading the SASS wherever nvcc was faster found three things ptxas does not do. In each case nvcc's NVVM stage does it before writing PTX.

1. **Inlining.** ptxas keeps `.func` calls as real `CALL` instructions, even at `-O3`. Inlining took device_fn.kc from 36 SASS instructions with 2 calls to 19 with none. The kernel is memory bound, so its time barely changed, but call overhead matters for compute-bound code.
2. **Loop unrolling.** ptxas keeps one load per loop iteration, so each iteration waits for memory. nvcc's PTX was already unrolled 4x (plus a remainder loop) and ran 1.45-1.49x faster on row_sum and matmul. kc's unroller handles counted loops (`for (let i = C; i < n; i += 1)`), writing the main-loop test as `i < n && n - i > 3`, which cannot overflow.
3. **Division by constants.** ptxas compiles `div.s32 x, 2` into a full division routine: 25 SASS instructions in a test kernel against 8 for a shift. This made block_sum's `s / 2` cost about 20 instructions per loop round. kc now emits shifts, with a 3-instruction correction for negative numbers (division rounds toward zero, a shift rounds down) unless it can prove the value is not negative. That proof handles loop variables by induction, so block_sum gets a single shift like nvcc.

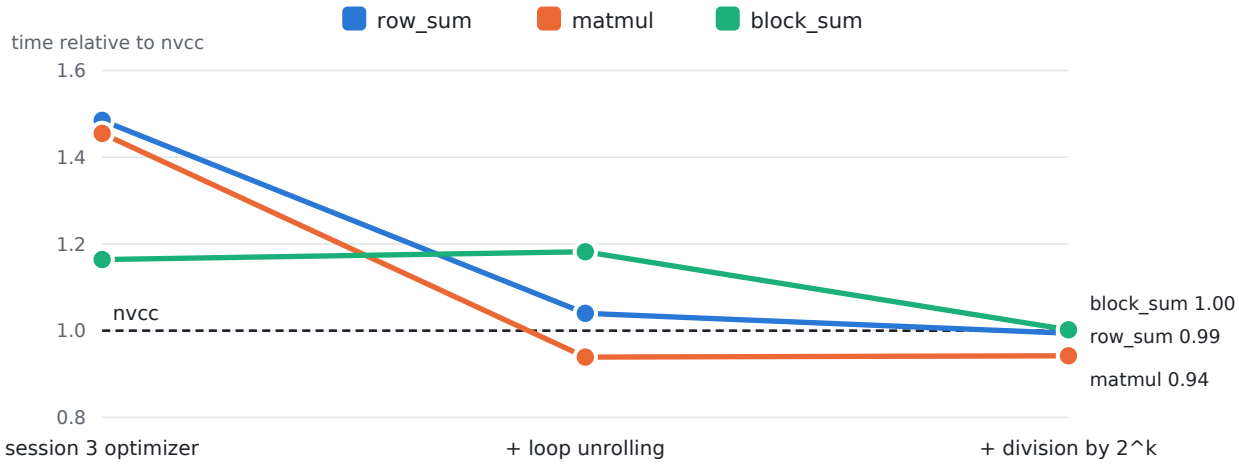


Figure 2. The three kernels where nvcc was faster, relative to nvcc in the same run, after each fix (start: row_sum 1.49, matmul 1.46, block_sum 1.16). Unrolling closed the gap for row_sum and matmul; strength reduction closed it for block_sum.

example	SASS -O0	SASS -O1	SASS nvcc	registers -O1
device_fn	36 (2 calls)	19	18	10
reduce (block_sum)	72	35	35	10
row_sum	23	56 (unrolled)	128	15
matmul	58	103 (unrolled)	217	23
saxpy	15	15	15	10
histogram	35	35	35	14
warp_reduce	22	22	27	8

4.4 Separate compilation has a cost

With `export fn` / `extern fn`, files are compiled separately and linked with nvJitLink. The same kernel then needed **24 registers instead of 10**, and the call stayed a real call: when ptxas compiles the kernel alone, it cannot see the callee, so it cannot inline it or know its

register needs, and has to follow the calling convention. Link-time optimization (nvcc's `-dlto`) exists for exactly this. Even the linked cubin still contains relocations (`CALL.ABS 0x0`) that the driver fills in when it loads the code into GPU memory.

4.5 A ptxas bug found by random testing

One of the 2,050 random programs gave different results at `-O0` and `-O1`. Two checks pointed away from kc. First, switching off kc's passes one at a time showed only `simplify` mattered. Second, with ptxas's own optimizer off, kc's `-O0` and `-O1` PTX agreed, so they meant the same thing. Shrinking the 70-line program by hand ended at six PTX instructions:

```
abs.s32      %r2, %r1;          // x = |x[0]|
neg.s32      %r3, %r2;          // -|x[0]|
setp.ge.s32  %p1, %r3, -5;      // -|x[0]| >= -5, true when |x[0]| <= 5
```

ptxas (13.3, at `-O3`, for every architecture from sm_75 to sm_90, and also the 13.1 driver's JIT) decides the comparison is always false. The SASS stores 0 without even loading `x[0]`, and the GPU returns false for x[0] = 0, 3 and -4. ptxas `-O0` gets it right. kc's `-O1` triggers it because it folds the right-hand side into a constant.

nvcc users do not run into it, because nvcc's front end rewrites `-abs(v) >= -5` into `abs(v) < 6` before writing PTX. C allows that rewrite because `-INT_MIN` is undefined behavior in C; in kc, integer overflow is defined to wrap, so kc cannot make the same rewrite. This is a concrete case of why PTX from producers other than NVVM is worth testing: it reaches paths in ptxas that nvcc's output never does. The reproducer is `tests/ptx/ptxas_neg_abs.ptx`, and kc-fuzz now sorts mismatches into "kc bug" and "suspected ptxas bug" automatically, using the ptxas-optimizer-off check.

5. Correctness

- **33 end-to-end tests**, each run at both `-O0` and `-O1` on the GPU and compared with a CPU reference. They include the hard cases: the phi "swap problem", loop-carried values, the inliner's corner cases, warp operations compiled for both sm_89 and sm_75, and separate compilation.
- **Bit-exact constant folding:** a test computes overflow, shifts by 32 or more, float-to-int clamping, negative zero and infinity both on the GPU (`-O0`) and folded on the CPU (`-O1`), and compares the bits.
- **Random differential testing:** kc-fuzz generated 2,050 random integer kernels (loops, `break` / `continue`, divisions, shifts, edge values) and ran each at `-O0` and `-O1` on the GPU. It found no kc bugs. The one mismatch was a ptxas bug (section 4.5).
- **Verifier and sanitizers:** the SSA verifier runs after every pass; compute-sanitizer memcheck and racecheck report no errors. With `-lineinfo`, compute-sanitizer reports a planted out-of-bounds write at the exact kc source line.

6. Limitations

The benchmark covers seven small kernels on one GPU, so it shows where the boundary lies, not that kc matches nvcc in general. kc has no alias analysis (CUDA's `__restrict__` would allow more load reuse), divides by other constants with real divisions (nvcc multiplies by a "magic number" instead), unrolls only simple counted loops, and has no link-time optimization.

7. Conclusion

Most classic SSA optimizations are redundant in front of ptxas, which already does them. A PTX producer gets the most out of a few things ptxas leaves alone: inlining, loop unrolling and division by constants. I found these by reading the SASS rather than assuming, and with them a 4,700-line compiler matches nvcc on the kernels measured here. Testing from the PTX side also pays off in the other direction: a small independent PTX producer found a real ptxas miscompilation.